```
template<typename T>
class lock_free_stack
{
private:
    void increase_head_count(counted_node_ptr& old_counter)
    {
        counted_node_ptr new_counter;

        do
        {
            new_counter=old_counter;
            ++new_counter.external_count;
        }
        while(!head.compare_exchange_strong(old_counter,new_counter)); <-1

        old_counter.external_count=new_counter.external_count;
    }

public:
    std::shared_ptr<T> pop()#
    {
        counted_node_ptr old_head=head.load();
        for(;;)
        {
            increase_head_count(old_head);
            node* const ptr=old_head.ptr;     <-2
            if(!ptr)
            {


                return std::shared_ptr<T>();
            }
            if(head.compare_exchange_strong(old_head,ptr->next))     <-3
            {
                std::shared_ptr<T> res;
                res.swap(ptr->data);     <-4

                int const count_increase=old_head.external_count-2;     <-5

                if(ptr->internal_count.fetch_add(count_increase)==     <-6
                   -count_increase)
                {
                    delete ptr;
                }

                return res;     <-7
            }
            else if(ptr->internal_count.fetch_sub(1)==1)
            {
                delete ptr;     <-8
            }
        }
    }
};
```